Experiment No-25 : Monkey Banana Problem

Aim:

To write a Prolog program to implement the Monkey Banana problem, in which a monkey must find a sequence of actions to reach a banana hanging from the ceiling.

Programming Code:

```
% state(MonkeyPosition, MonkeyOn, BoxPosition, HasBanana)
% The banana hangs from the ceiling in the middle of the room.

position(atdoor).
position(atwindow).
position(middle).

% move(State1, Action, State2)
% 1. Grasp the banana (monkey on the box, in the middle of the room)
move(state(middle, onbox, middle, hasnot), grasp,
     state(middle, onbox, middle, has)).
% 2. Climb on the box (monkey and box at the same place)
move(state(P, onfloor, P, H), climb, state(P, onbox, P, H)).
% 3. Push the box from P1 to P2
move(state(P1, onfloor, P1, H), push(P1, P2), state(P2, onfloor, P2, H)) :-
    position(P2), P1 \= P2.
% 4. Walk from P1 to P2
move(state(P1, onfloor, B, H), walk(P1, P2), state(P2, onfloor, B, H)) :-
    position(P2), P1 \= P2.

% plan(State, VisitedStates, ListOfActions)
plan(state(_, _, _, has), _, []).
plan(S1, Visited, [Action|Rest]) :-
    move(S1, Action, S2),
    \+ member(S2, Visited),
    plan(S2, [S2|Visited], Rest).

% solve(Start, Plan): shortest plan (tries plans of 0 to 10 moves)
solve(Start, Plan) :-
    between(0, 10, N),
    length(Plan, N),
    plan(Start, [Start], Plan), !.

% Print the plan step by step
show_solution(Start) :-
    solve(Start, Plan),
    print_steps(Plan, 1),
    length(Plan, L),
    format("The monkey gets the banana in ~w steps.~n", [L]).

print_steps([], _).
print_steps([A|T], N) :-
    format("Step ~w: ~w~n", [N, A]),
    N1 is N + 1,
    print_steps(T, N1).
```

OUTPUT:

```
$ swipl -q exp25.pl

?- solve(state(atdoor, onfloor, atwindow, hasnot), Plan).
Plan = [walk(atdoor,atwindow),push(atwindow,middle),climb,grasp].

?- show_solution(state(atdoor, onfloor, atwindow, hasnot)).
Step 1: walk(atdoor,atwindow)
Step 2: push(atwindow,middle)
Step 3: climb
Step 4: grasp
The monkey gets the banana in 4 steps.
true.

?- solve(state(middle, onbox, middle, hasnot), Plan).
Plan = [grasp].

?- solve(state(atwindow, onfloor, atdoor, hasnot), Plan).
Plan = [walk(atwindow,atdoor),push(atdoor,middle),climb,grasp].

?- move(state(middle, onbox, middle, hasnot), grasp, S).
S = state(middle,onbox,middle,has).

$
```

Result

The Monkey Banana problem was implemented successfully using Prolog. The program searches the states of the problem and finds the shortest sequence of actions (walk, push, climb and grasp) that lets the monkey get the banana.